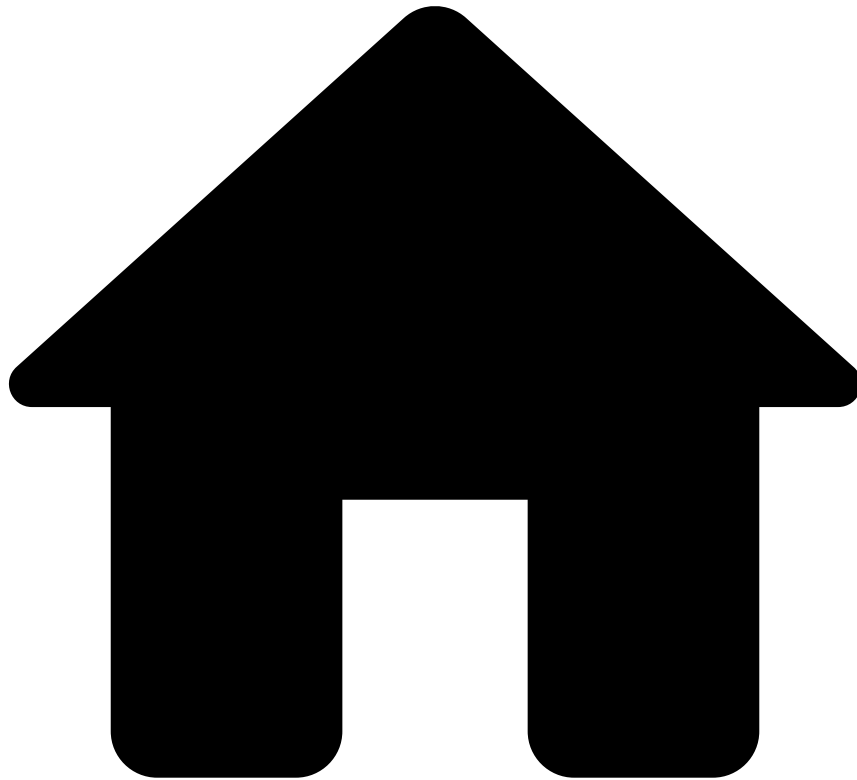


-



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding models

On this page

Adding models

Was this page helpful? 

Why

Similarly to [Workflow Plans](#), Pulse lets you take the **Model** definition that yielded the best results during the Data Science stage directly into Production without significant effort.

Once you obtain a [Model Evaluation](#) that shows that your **Model** configuration has reached or surpassed business objectives, you can simply apply it in runtime.

The **Scoring Service** in a **Workflow** runs events through the Machine Learning algorithm, by default Feedzai Random Forest (FZRF), with the exact same parameter configuration as tested during the Data Science stage.

How to

To add and configure a [Model](#) to score events in your **Workflow**, execute the following actions:

1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Scoring Service** to the **Diagram** canvas.
3.  **Edit** the **Scoring Service**.
4. In the **Main** tab:
 1. Define to which of the [Workflow Outcomes](#) the **Model** will contribute. After being processed by a **Scoring Service**, the `<outcome_name>_and_<outcome_name>_score` fields in the event schema will reflect the **Model** decision.
 2. Select the **Model** you want to apply in your runtime processing.
 3. Optionally, define a **Filter** to narrow down events that are scored in your **Workflow**. Filter conditions are written in PulseQL.



Fields used in Filter

You can only use fields in the **Filter** which were present in the [data source](#) you used to train the **Model**.

4. Define how much **Weight** the **Model** results will have in the final decision, when the event arrives at the end of the **Workflow**. For example, you can have real-time events flowing through different **Scoring Services** and [Rules Services](#). These scores need to be combined to produce a final score for the event.
5. White-Box Explanations that were added with the new model in a first instance (see section **Training parameters** in [Training a Model](#)) are then visible in the **Scoring Service** and automatically used.



Migrating old White-Box Explanations

After upgrading your Pulse version to **Pulse 20.1**,

you may need to reconfigure the old WBE. This is in case you enabled the old **Feedzai Random Forest Feature Contribution**, which only generated contribution numbers and could not work with natural language sentences. To do so, **Edit** the **Scoring Service** and follow the instructions in the UI.

Furthermore, if you were using the old **Feature Contribution** for **Model Explanations**, then you were most likely using a Custom extended **Alert Storage Service**. In that case, please check **Case Manager 8.1** release notes for more information as you no longer need to do that.



Explanations on critical path

True â Explanations are calculated in the **Scoring element** and put in the event.

False â Explanations are only calculated in the **Dead Letter** (i.e. at the end of the workflow).

5. In the **Schemas** tab:

1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Model**.
2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.
6. â **Connect** the previous service in the **Workflow** to the Scoring Service.

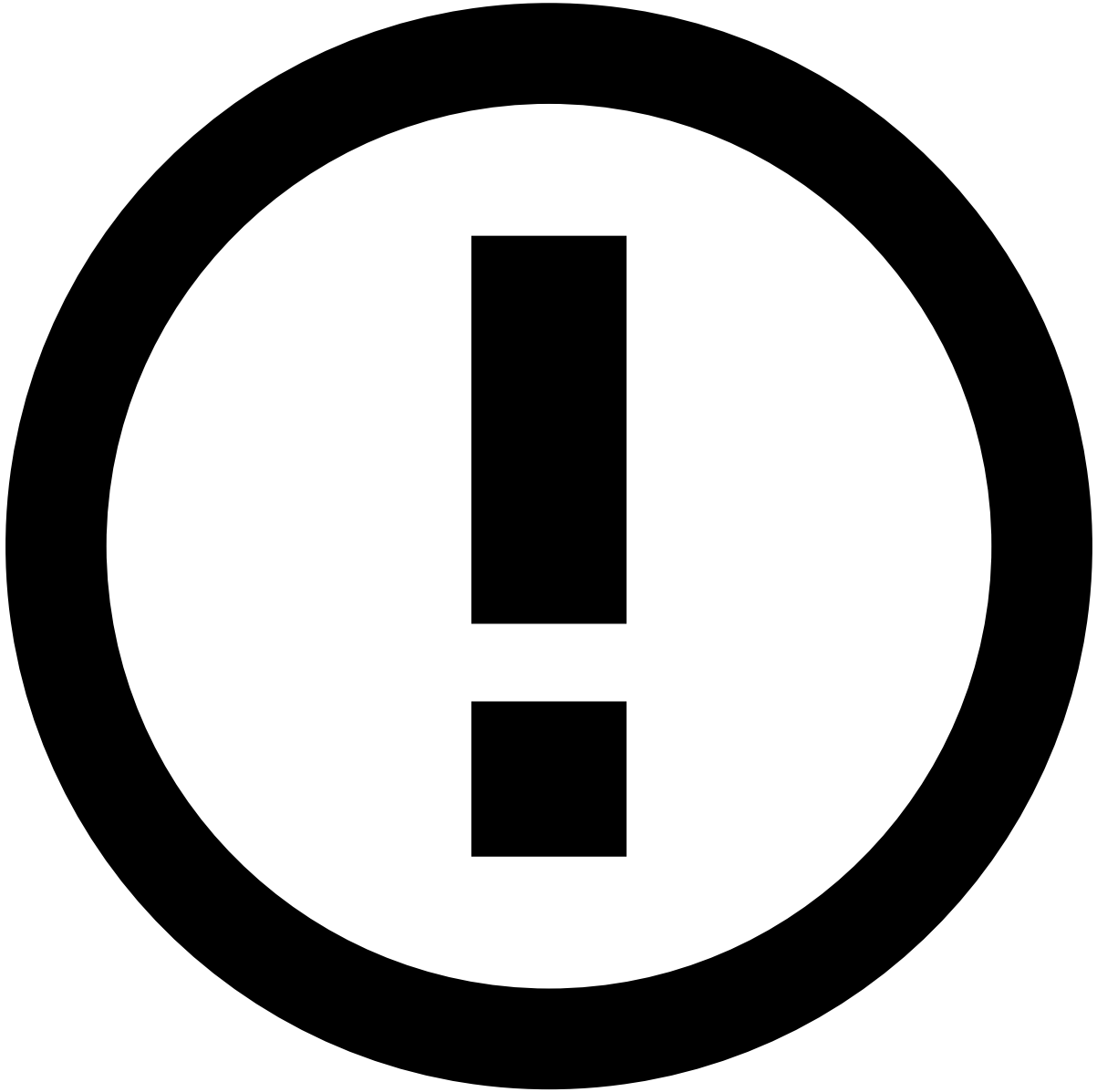
Solving schema conflictsâ

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

1. â Edit the **Service**.
2. Select the **Schemas** tab.
3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.

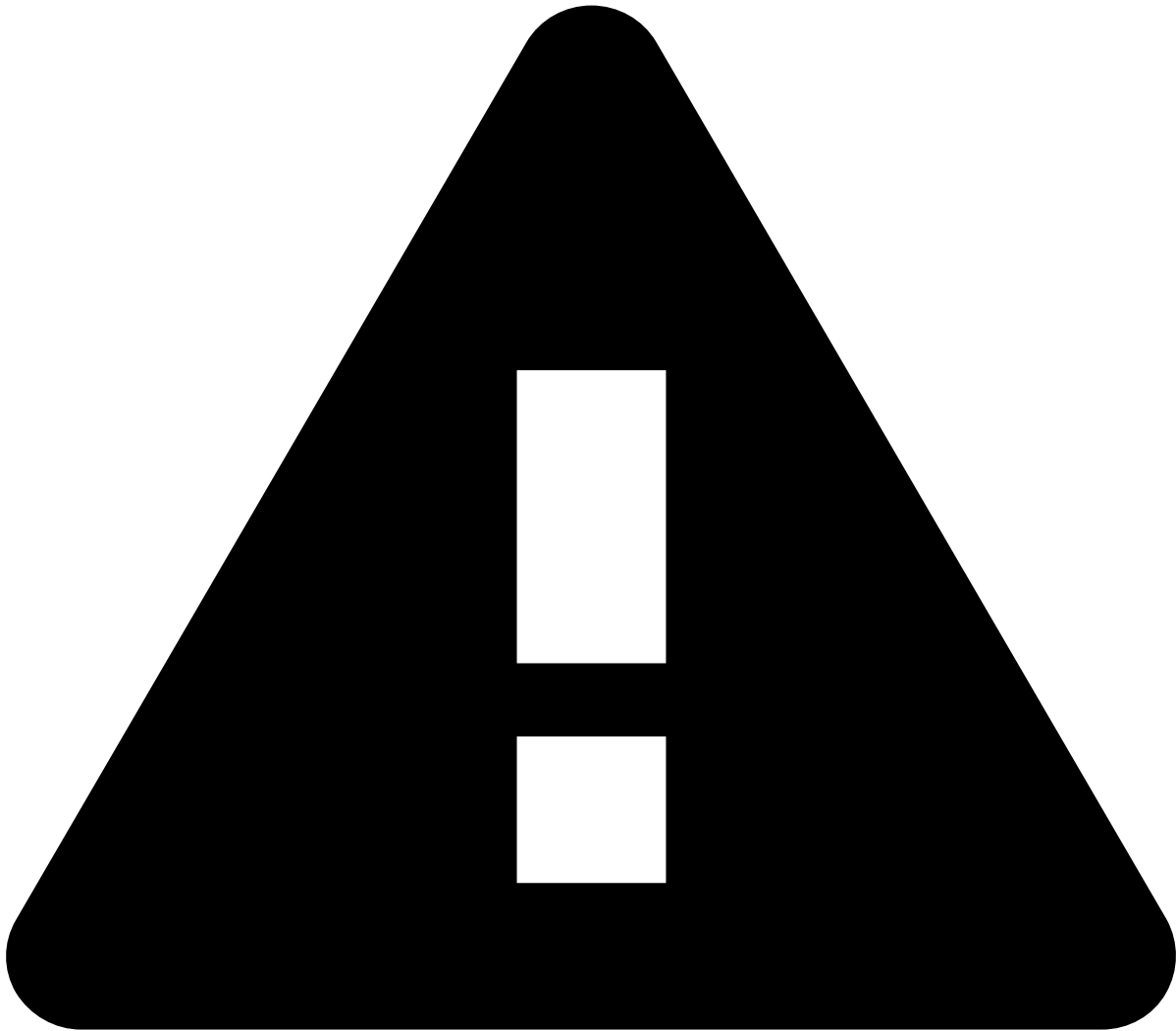
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Next Steps📖

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding plans](#)
- [Adding rules](#)
- [Adding custom services](#)